對外秘

프로젝트 명세서

금융 자산 정보 토론 게시판

(Python)

“금융 상품 비교 애플리케이션”

4 회차

對外秘

목차

1. 금융 자산 정보 토론 게시판 3

1.1 목표 3

1.2 준비사항 3

1.3 작업 순서 4

1.4 요구사항 4

1.5 참고자료 16

1.6 프로젝트 결과 16

對外秘

PJT 명	금융 자산 정보 토론 게시판	
단계	04 PJT	
진행일자	2026.04.17	
예상 구현 시간	필수기능	6H
	심화기능	2H

1. 금융 자산 정보 토론 게시판

1.1 목표

이번 프로젝트의 목표는 다음과 같습니다.

- Django의 URL 라우팅, View, Template, Model을 활용해 CRUD 흐름을 이해한다.
- JSON 파일 기반의 초기 데이터를 읽어 웹 화면에 출력하는 방법을 익힌다.
- 금융 자산별 게시판 구조를 설계하고 게시글 작성/조회/수정/삭제 기능을 구현한다.
- 예외 상황(잘못된 자산 ID 등)에 대한 404 처리로 안정적인 사용자 경험을 제공한다.
- (심화) LLM API 를 활용해 게시글의 부적절 표현을 사전 차단하는 콘텐츠 필터링을 구현한다.

1.2 준비사항

1) 개발언어 및 툴

- Python, Django
- JSON
- SQLite
- python-dotenv(환경변수 관리)
- Visual Studio Code
- OpenAI 또는 Upstage API (심화)

1.3 작업 순서

1) 페어와 같이 요구사항(기본/심화)을 분석한다.
2) 페어와 함께 개발 시 역할을 분담한다.
3) 팀장/네비게이터 등 역할 분담
4) [선택] 추가적인 아이디어를 적용하여 구현하고 프로젝트를 완성한다.
5) 코드 및 산출물을 정리하여 Git 에 반영한다.

1.4 요구사항

본 프로젝트는 금융 자산 목록 데이터를 기반으로 자산별 토론 게시판을 제공하는 Django CRUD 웹 애플리케이션 구현을 목표로 한다. data/assets.json 파일에서 자산 정보를 로드하여 목록 화면에 출력하고, 각 자산 상세 페이지에서 게시글 CRUD 기능을 수행할 수 있도록 구성한다.

이를 통해 파일 데이터 연동, 데이터베이스 기반 웹 개발, 예외 처리(404) 등 Django 핵심 개발 흐름을 종합적으로 학습할 수 있다. 추가적으로 LLM API 를 연계하여 게시글 등록/수정 시 욕설·비방 등 부적절한 내용을 필터링함으로써, 실무형 AI 기능 확장 가능성까지 경험할 수 있다.

■ **요구사항 예시 (참고용)**

- 아래의 내용을 참고하여 추가적인 아이디어에 대해 요구사항을 추가 또는 수정하여 기능을 구현한다. 단, 필수 기능은 반드시 구현해야 하며 수정할 수 없다.

기능적 요구사항				
번호	분류	요구사항명	요구사항 상세	우선순위
F401	자산 데이터	자산 목록 출력하기	제공된 assets.json 파일을 읽어 자산 목록 페이지에 출력한다. 각 항목에는 자산명, 코드(또는 식별자), 게시판 이동 링크를 포함한다.	필수
F402	모델 설계	게시판 모델 작성	게시글 저장을 위한 모델(Post)을 정의한다. 세부 스키마는 상세 요구사항에서 제공된다.	필수
F403	게시판 화면	자산별 게시판 구현	URL 파라미터의 asset_id를 기준으로 해당 자산 게시판을 제공한다. 자산	필수

			정보와 함께 해당 자산의 게시글 목록을 출력한다.	
F404	게시글 관리	게시글 생성	게시글 작성 폼을 제공하고, 유효한 입력값(제목/내용)을 받아 DB에 저장한다. 저장 후 해당 자산 게시판 목록으로 리다이렉트한다.	필수
F405	게시글 관리	게시글 상세 조회	게시글 목록에서 특정 게시글 선택 시 상세 페이지를 제공한다. 제목, 내용, 작성자, 작성일 정보를 표시한다.	필수
F406	게시글 관리	게시글 수정	기존 게시글 데이터를 폼에 바인딩하여 수정 가능하게 한다. 수정 완료 시 DB 반영 후 상세 페이지로 이동한다.	필수
F407	게시글 관리	게시글 삭제	삭제 요청(POST) 시 해당 게시글을 DB에서 제거한다. 삭제 완료 후 자산별 게시판 목록으로 이동하고 안내 메시지를 표시한다.	필수
F408	예외 처리	잘못된 경로/ID 처리	존재하지 않는 asset_id 또는 post_id 접근 시 404 상태 코드를 반환한다. 사용자에게 오류 안내 페이지(또는 기본 404 페이지)를 제공한다.	필수
F411	환경 설정	환경변수 설정	.env의 MODE(OPENAI/UPSTAGE) 및 API 키를 읽어 공급자별 호출 처리, 실패 시 안전하게 우회	심화
F412	콘텐츠 검증	LLM 부적절 표현 필터링	게시글 등록/수정 시 욕설·비방 등 부적절 문구를 LLM으로 검사 후 차단	심화
…	…	…	…	…

비기능적 요구사항				
번호	분류	요구사항명	요구사항 상세	우선순위
NF401	문서화	README 작성	분석 흐름과 학습 내용, 어려웠던 점을 README.md에 기록하여 제출	필수
NF402	유지보수	코드 정리	.gitignore를 추가하여 불필요한 파일 제외	필수
…	…	…	…	…

1. 기본 기능 (필수)

본 프로젝트는 금융 자산 정보를 기반으로 자산별 토론 게시판을 구축하고 게시글을 관리할 수 있는 웹 애플리케이션 구현을 목표로 한다. JSON 파일의 자산 데이터를 불러와 목록 및 자산별 게시판 화면을 제공하고, 게시글의 조회 · 상세 조회 · 수정 · 삭제 기능을 지원한다. 또한 존재하지 않는 자산 또는 게시글 접근 시 404 예외 처리를 통해 안정적인 사용자 경험을 제공한다.

요구사항의 기능만 동일하게 동작한다면, 디자인은 자유롭게 구성하는 걸 권장한다.

1) 자산 목록 출력하기

① 요구사항 번호: F401

② assets.json 으로부터 금융 자산 데이터를 읽어온다.

③ 읽어온 자산 목록을 페이지로 표시한다.

- ◆ 각 자산은 해당 자산에 대한 토론 게시판으로 이동 가능한 링크를 제공한다.

④ 자산에 대한 요약 정보를 출력한다.

<그림 1>금융 자산 목록 출력 예시

2) 게시판 모델 작성

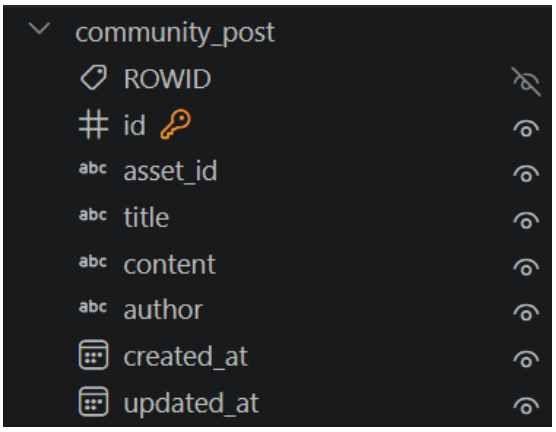
① 요구사항 번호: F402

② 게시글 저장을 위한 모델 Post 를 정의한다.

③ 기본 스키마는 아래와 같다.

필드명 (Field)	타입 (Type)	제약조건 (Constraints)	설명 (Description)
asset_id	CharField	max_length=50	금융 자산 ID
title	CharField	max_length=200	게시글 제목
content	TextField	X	게시글 내용
author	CharField	default="익명"	게시글 작성자
created_at	DateTimeField	자동 생성	생성 일자
updated_at	DateTimeField	자동 갱신	수정 일자

④ 마이그레이션 명령어를 통해 데이터베이스에 반영한다.



<그림 2> 마이그레이션 후 적용된 DB 스키마 예시

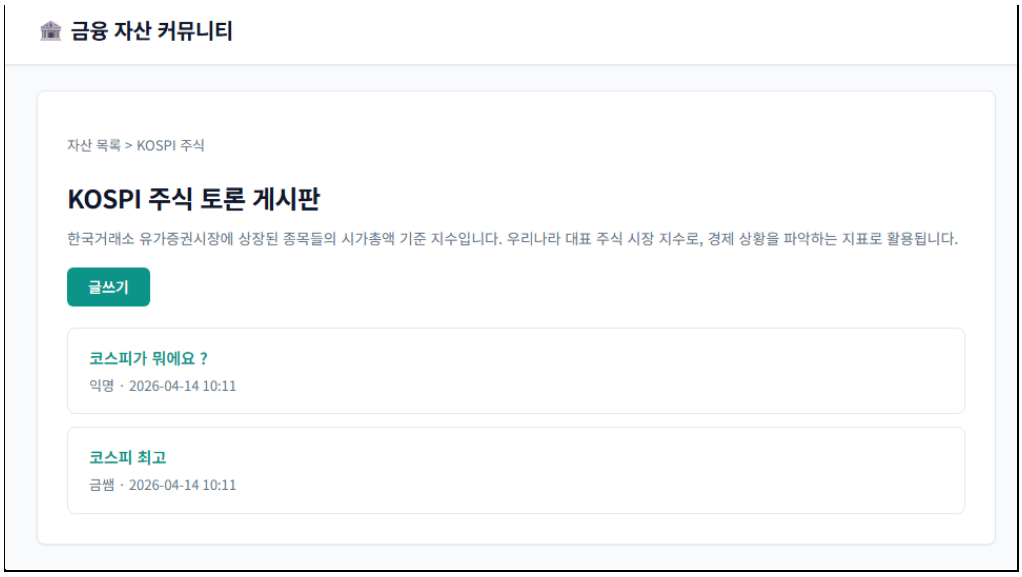
3) 자산 별 게시판 구현하기

① 요구사항 번호: F403

② 선택한 자산의 id 등을 기준으로 해당 자산의 토론 게시판 페이지를 제공한다.

③ 선택한 자산에 대한 정보(이름/코드 등)와 관련 게시글 목록을 함께 출력한다.

- ◆ 게시글은 최신순으로 정렬하여 출력한다.
- ◆ 게시글 제목, 작성자, 작성일 등의 요약 정보를 표시한다.



<그림 3> 금융 자산 토론 게시판 예시

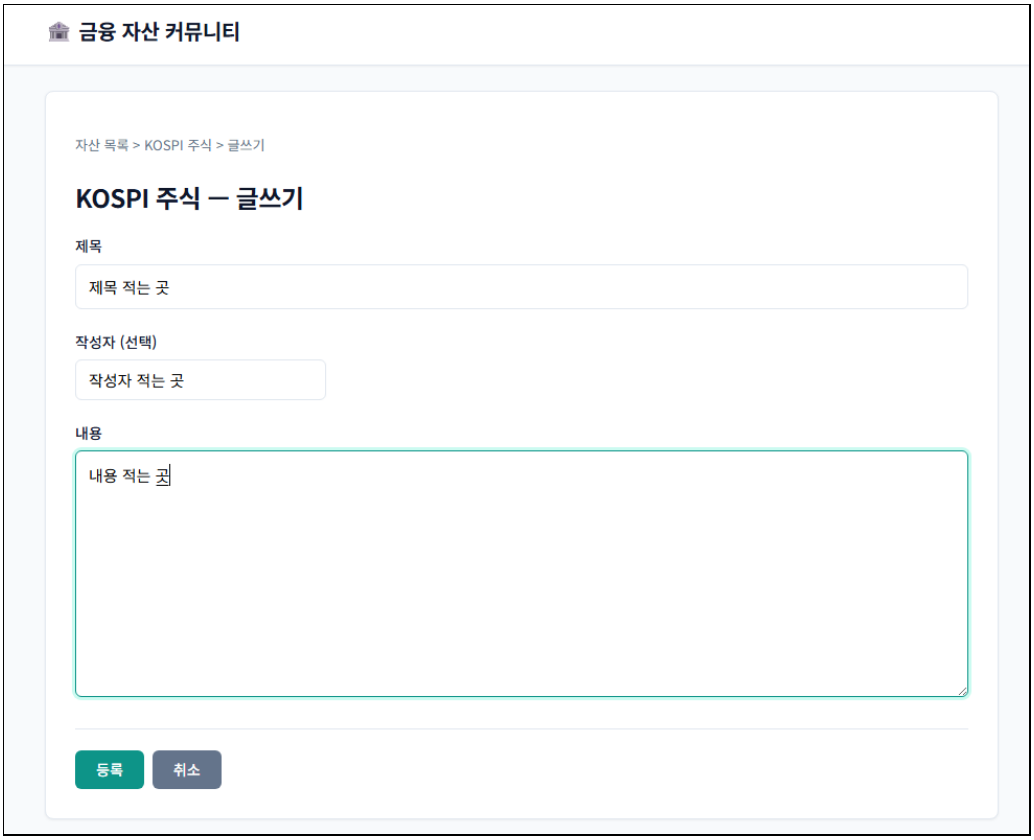
4) 게시글 생성

① 요구사항 번호: F404

② 사용자로부터 게시글에 대한 정보를 입력받을 수 있는 생성 페이지를 제공한다.

③ 제목, 내용에 유효한 입력값을 받았을 때만 DB에 저장한다.

④ 저장 후 해당 자산 게시판 목록으로 리다이렉트한다.



<그림 4> 게시글 생성 페이지 예시

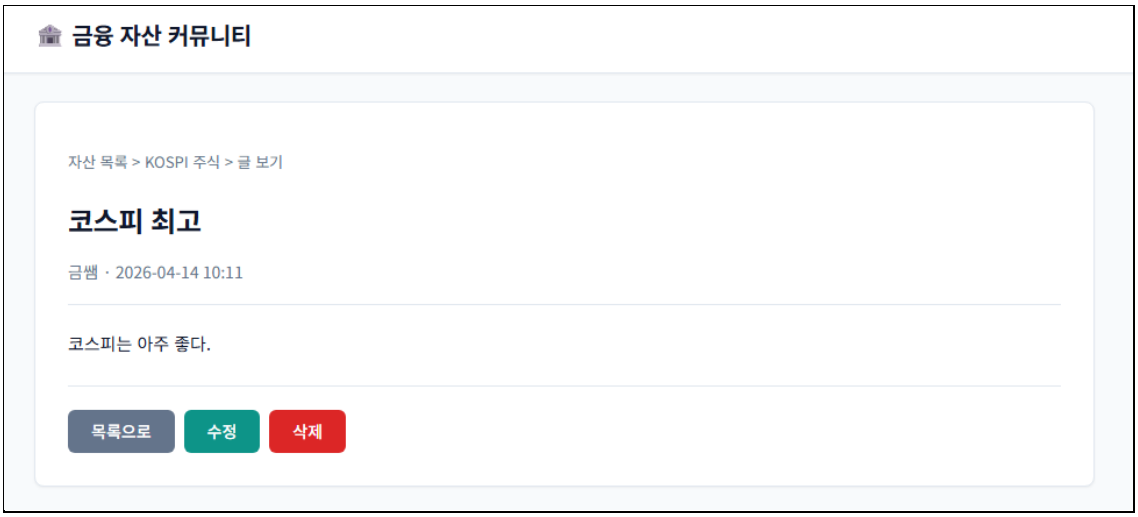
5) 게시글 상세 조회

① 요구사항 번호: F405

② 게시글 목록에서 특정 게시글 선택 시 상세 페이지로 이동한다.

③ 제목, 내용, 작성자, 작성일 등 게시글 전체에 대한 내용을 출력한다.

- ◆ 수정 기능 (F406 에서 구현)
- ◆ 삭제 기능 (F407 에서 구현)



<그림 5> 게시글 상세 페이지 예시

6) 게시글 수정

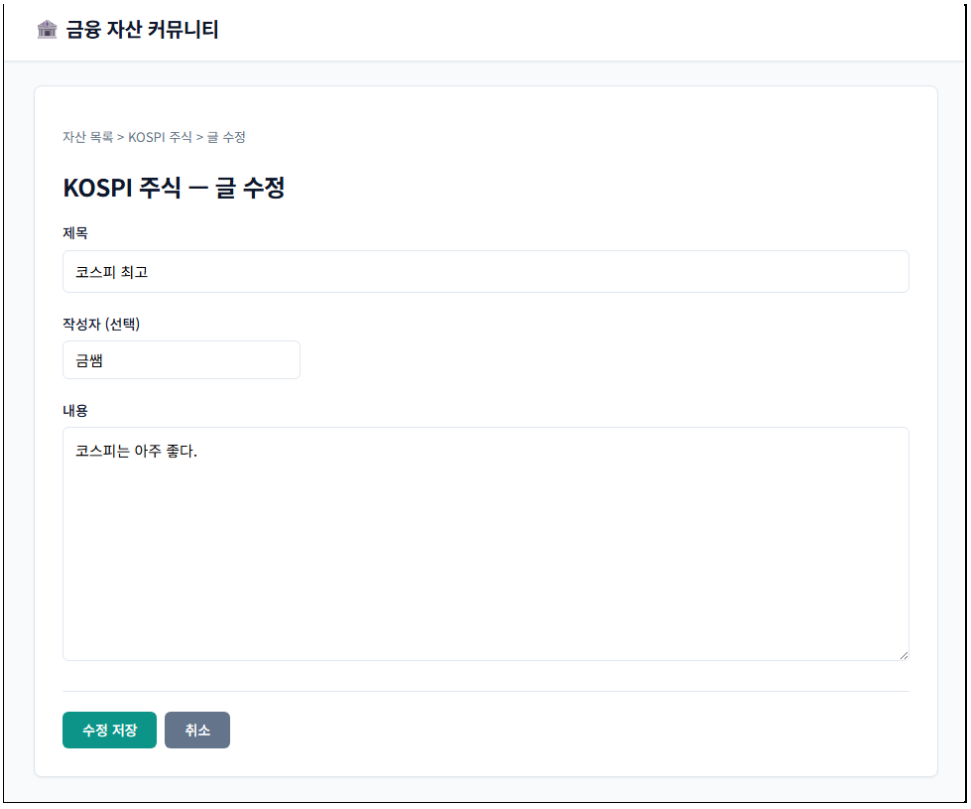
① 요구사항 번호: F406

② F405 에서 구현한 게시글 상세 페이지에 수정 버튼을 추가한다.

③ 기존 게시글의 제목, 내용, 작성자 정보를 사용자로부터 입력받고, 해당 내용으로 수정할 수 있도록 구성한다,

◆ 모든 필드에 유효한 입력값일 때만 DB 에 반영한다.

④ DB 반영 후 해당 게시글 상세 페이지로 이동한다.



<그림 6> 게시글 수정 페이지 예시

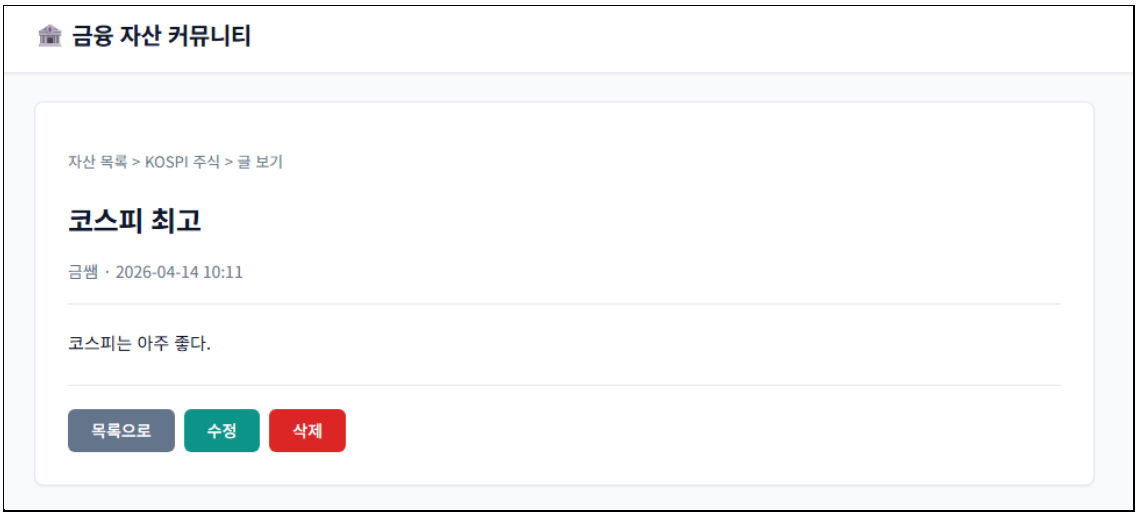
7) 게시글 삭제

① 요구사항 번호: F407

② F405 에서 구현한 게시글 상세 페이지에 삭제 버튼을 추가한다.

③ 삭제 요청(POST) 시 해당 게시글을 DB 에서 제거한다.

④ 삭제 완료 후 해당 자산 게시판 목록으로 이동한다.

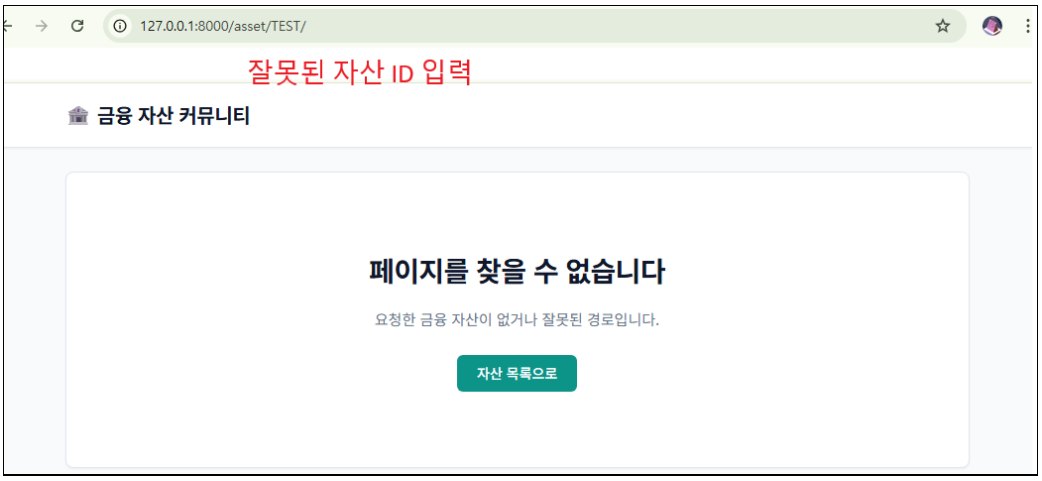


<그림 7> 게시글 상세 페이지 예시

8) 잘못된 경로/ID 예외 처리

① 요구사항 번호: F408

② 존재하지 않는 자산 ID 또는 게시글 ID 로 접근 시 사용자에게 오류 안내 페이지 (404.html) 을 제공한다.



<그림 8> 404.html 출력 예시

2. 심화 기능 (선택)

본 심화 기능은 OpenAI API 또는 Upstage API 를 통해 부적절한 내용에 대한 검열 기능을 추가한다.

아래 요구사항은 LLM API 호출 시 OpenAI API 혹은 Upstage API 둘 중 하나를 선택 후 진행해야 한다.

1) 환경 변수 설정

① 요구사항 번호: F411

② .env 파일을 생성하고 아래 환경변수를 작성한다.

- ◆ UPSTAGE_API_KEY - Upstage API 활용 시 작성
- ◆ OPENAI_API_KEY - OPENAI API 활용 시 작성

③ Django 프로젝트가 해당 환경변수를 읽어올 수 있도록 settings.py 에 작성한다.

2) LLM 부적절 표현 필터링

① 요구사항 번호: F412

② 게시글 생성/수정 시 욕설, 비방, 혐오 등 부적절한 문구를 LLM 으로 검사하고 차단할 수 있도록 구성한다.

◆ 제목(title) / 내용(content)에 대해 적용한다.

③ 차단 방식은 자유롭게 구성한다.

◆ 부적절 컨텐츠 검열 문구 출력 / 다른 페이지로 이동 / 자동 수정 등

<그림 9> 부적절한 내용 필터링 후 검열 문구 출력 화면 예시

1.5 참고자료

1) Django 공식 문서
   - https://docs.djangoproject.com/
2) OpenAI API 문서 (심화 기능용)
   - https://platform.openai.com/docs/
3) Upstage API 문서 (심화 기능용)
   - https://console.upstage.ai/docs/getting-started

1.6 프로젝트 결과

최종적으로 제출해야 할 항목은 아래와 같다

■ **산출물과 제출**

1) 구현 소스 코드

◆ Django 프로젝트 전체 소스 코드

2) README.md

◆ 구현 기능 설명, 학습 내용, 느낀 점 포함

3) 위 내용을 모두 포함하여 GitLab에 업로드한다.

◆ GitLab 프로젝트 이름은 프로젝트 번호 + pjt 형식으로 지정한다 (04_pjt)

- 以上